



22.15/25.30 Electrónica V

Trabajo Práctico

Juan Pablo Cilfone Argibay - Legajo: 62074

Bruno Di Sanzo - Legajo: 62497

Candela Gioia - Legajo: 62447

Lucía Ibañez - Legajo: 62488

Agustín Solari Raigoso - Legajo: 62626

Ingeniería Electrónica

Índice

1. Introducción	2
1.1. Implementación general	2
1.2. Predictor de saltos: mejora propuesta	3
1.2.1. Estructura de la tabla	4
1.2.2. Decisión de predicción	4
1.2.3. Actualización de un registro del predictor	5
2. Implementación detallada del proyecto	7
2.1. Fetch Stage	7
2.2. Decode Stage	7
2.2.1. Opcode Decode	8
2.2.2. Immediate builder	9
2.2.3. Funct3, Funct7, RS1, RS2 y RD	9
2.3. Operand y Address Builder Stage	9
2.3.1. Banco de registros en la etapa Operand	9
2.3.2. Address Builder	10
2.4. Execute, Memory y PC update Stage	10
2.4.1. Execute: ALU y Operand Build	11
2.4.2. Memoria: store_data_module	11
2.4.3. PC Update: Next_pc.decision_module y Branch_result	12
2.5. Write Back	13
2.5.1. Load Data Module	13
2.5.2. Banco de registros en Write Back	13
2.6. Pipeline Control	14
2.6.1. Funcionamiento del Pipeline Control	14
3. Simulaciones	16
3.1. Compilation Report	16
3.2. Simulación con código de Fibonacci	17
3.2.1. Simulación con el predictor de saltos activado	17
3.2.2. Simulación sin el predictor de saltos activado	18
4. Prueba en FPGA	20
4.1. Código de prueba: contador de números primos	20
4.2. Resultados para el código con el predictor de saltos activado	21
4.3. Resultados para el código sin el predictor de saltos activado	21
4.4. Conclusiones	22

1. Introducción

En el presente informe, se detallará la implementación del procesador ERV25, con su set de instrucciones RISC-V RV32I simplificado, en una FPGA. Se utilizó el programa Quartus Prime Lite Edition para realizar el desarrollo, aprovechándose su interfaz gráfica de bloques y programando cada bloque en el lenguaje Verilog.

Como implementación de mejora para este proyecto, se desarrolló un módulo de predictor de saltos. El mismo consta de un módulo que posee una tabla de memoria RAM, la cual es accedida de manera similar a una memoria caché, en la cual se almacenan, para cada PC correspondiente a una instrucción de salto, un contador de 2 bits que se incrementa o decrementa en función del resultado del último salto. La tabla almacena bits de validación, de tag y un campo destinado a almacenar la dirección de destino del salto, de manera de tomar el salto de manera inmediata si el predictor indica que debe tomarse el salto, en función del bit más significativo del contador.

1.1. Implementación general

Para la implementación, se basó fuertemente la estructura en la propuesta por la cátedra en la consigna. La misma se puede visualizar en la figura 1.

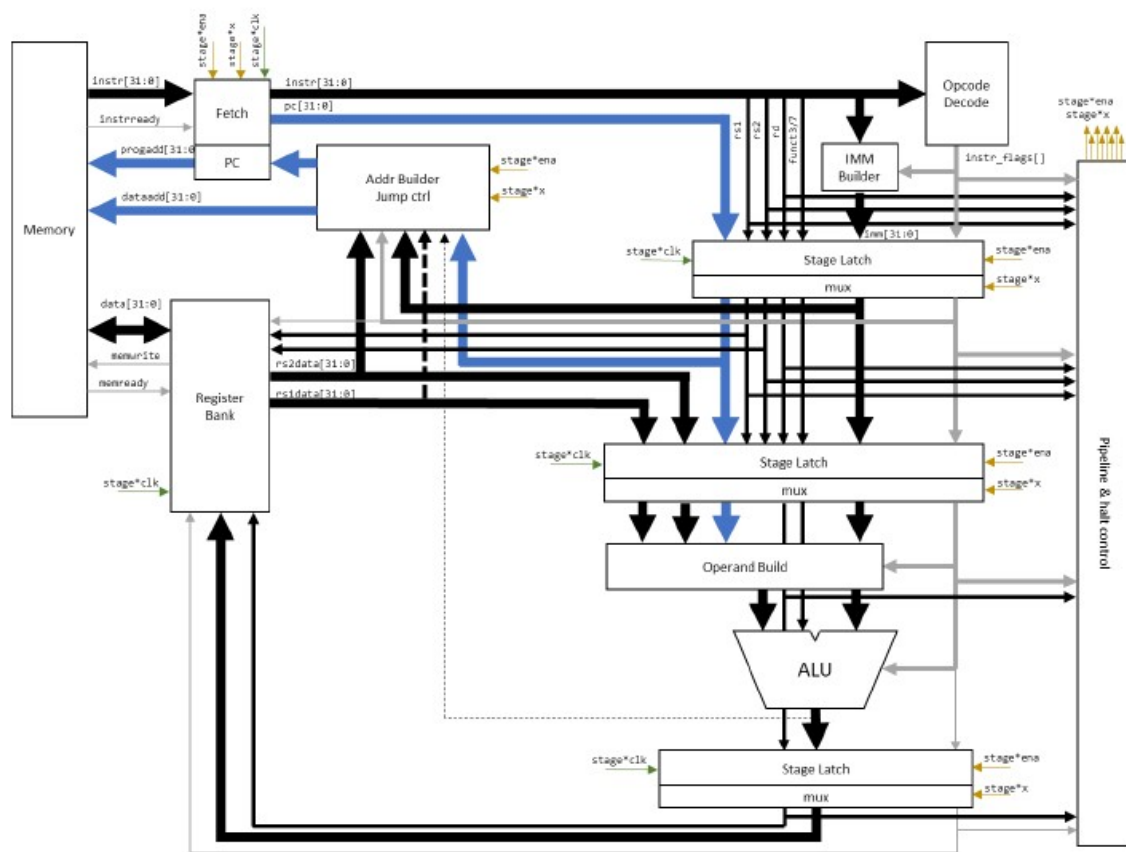


Figura 2.1 – Prototipo de arquitectura con pipeline de 5 etapas

Figura 1: Estructura del procesador en que se basó el desarrollo

En nuestra implementación, se pueden identificar 5 etapas:

1. **Fetch**
2. **Decode**
3. **Operand/Address Builder**
4. **Execute/Memory/PC Decision**

5. Write Back

La etapa **Fetch** se encarga de almacenar el PC y de recibir las instrucciones.

La etapa **Decode** se encarga de decodificar las instrucciones y de obtener todos los operandos necesarios y flags necesarios para que las siguientes etapas puedan saber qué operaciones realiza y cómo hacerlas.

La etapa **Operand/Address Builder** se encarga de leer los registros necesarios para la instrucción, y en caso de los saltos o de acceso a memoria (lectura o escritura), de calcular el address correspondiente (a memoria de datos o para calcular el PC siguiente).

La etapa **Execute/Memory/PC Decision** se encarga de ejecutar la instrucción a través de la ALU; se encarga de enviar el address correspondiente a memoria y, en caso de escritura, se envían los datos a guardar y una indicación de que se deben guardar; y en esta etapa también se decide cuál será el próximo PC, en función de si la instrucción es un salto, si hubo un error de predicción y también en función de lo que dice el predictor de saltos para el PC actual (de la instrucción que está en **Fetch**).

La etapa **Write Back** se encarga de decidir qué registro del banco de registros debe actualizarse, y en caso de deber hacerse, si debe actualizarse con el valor que salió de la ALU o con el valor que provino de memoria.

Es importante destacar que, con este diseño, se logra que todas las instrucciones accedan a memoria, para lectura o escritura, en la misma etapa del pipeline, de manera de evitar colisiones allí. También, permite que estén sincronizadas las escrituras a los registros, ya sea que vengan de la ALU o de memoria los datos, al escribirse en los mismo siempre en la etapa de "Write Back".

En caso de predicciones de saltos incorrectas o de haber colisiones entre instrucciones sucesivas en el acceso a registros que aún no fueron modificados debido a que aún no se llegó a la etapa de "Write Back", el Pipeline Control introduce los Halts, Nops y Flush de los latches del Pipeline necesarios para lograr un correcto funcionamiento y sincronismo en las instrucciones.

Adicionalmente, cabe aclarar que a lo largo del diseño, se intentó aprovechar la estructura de las instrucciones, de manera de identificar cómo instrucciones del mismo "formato" son procesadas de la misma manera. De esta manera, pudimos aprovechar el formato de instrucción para utilizar esto como señal indicadora de varios módulos de cómo deben operar. Así, se evitó tener que usar el "opcode" como entrada a varios módulos, ahorrando algunos bits.

Por último, es importante aclarar que este diseño permite que varios módulos realicen operaciones aún cuando la instrucción en cuestión no utiliza dichos módulos. Esto se puede lograr gracias a que toda operación que modifique a la memoria o a los registros posee un flag proveniente de la etapa de decode que indica si se debe realizar el "write" o la modificación del registro con los valores obtenidos. Por lo tanto, sin importar si los datos calculados tienen sentido o no, no se afectaran a la memoria o a los registros si el flag correspondiente no está activado. Esto, en principio, permite evitar el agregado de lógica adicional por cada bloque para que no realicen operaciones que no necesitan realizar; y permite además la posibilidad de agregar instrucciones de manera sencilla, simplemente modificando los módulos correspondientes a la etapa de Decode.

1.2. Predictor de saltos: mejora propuesta

A continuación, detallaremos cómo funciona nuestra mejora propuesta: el Predictor de Saltos. Para el funcionamiento del mismo, la lógica por defecto consiste en tomar la instrucción correspondiente al "pc+4" cuando no se predice negativamente, o tomar el salto cuando se predice positivamente. Así, nunca se realiza una detención del Pipeline. Recién cuando se llega a la etapa Execute/Memory/PC Decision, en la que se resuelve la condición de salto, se compara con la predicción, y si la misma fue incorrecta, se realiza un flush de las instrucciones posteriores al salto, que son incorrectas. La salida de este predictor está habilitada o deshabilitada por una señal de enable, para poder testear cómo sería un programa con el predictor o sin el predictor.

Este predictor de saltos posee una tabla en la cual se almacena un contador para cada PC a predecir. Dicho contador se aumenta en 1 cuando se toma el salto, y se disminuye en 1 cuando no se toma el salto. Así, obtiene una estadística básica de si es frecuentemente tomado o no (el contador satura en 00 y en 11). El salto se predice como "debe ser tomado" si el contador se encuentra en 10 o 11.

A la salida, el predictor también indica los 32 bits del "pc_target_prediction", que deja al alcance el PC al cual se debe saltar en el siguiente ciclo de clock, realizando la predicción sin delays. Como esta dirección es almacenada en la tabla, se asume que es siempre la misma dirección de salto para dicha instrucción. Por este motivo, este predictor de saltos sólo puede ser utilizado para saltos condicionales (que siempre saltan a la misma línea) o para saltos incondicionales que usen "pc + inmediato" y que no

utilicen el valor "pc + rs1", debido a que el contenido de los registros es variable, y daría lugar a un salto a una línea incorrecta.

En la figura 2 se puede observar el bloque del módulo en Quartus.

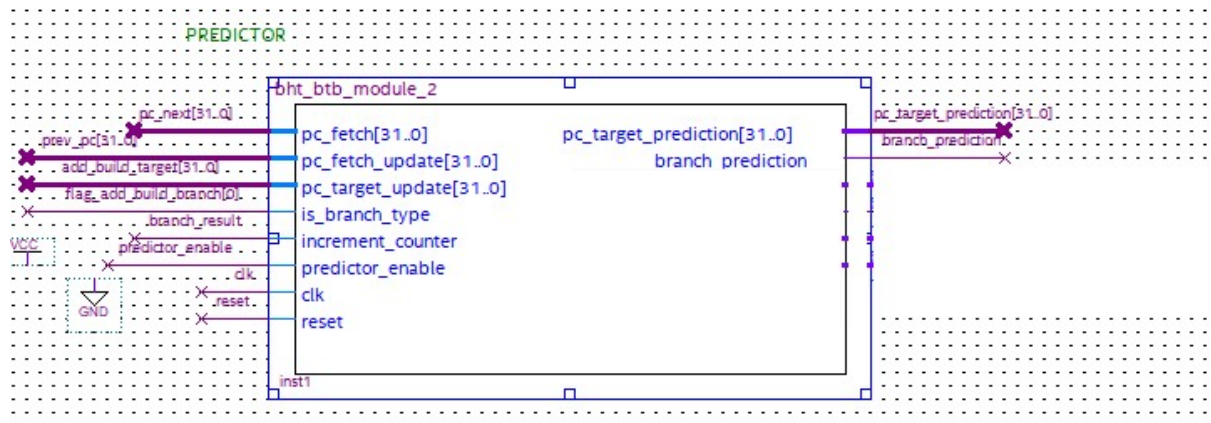


Figura 2: Bloque del predictor de saltos en Quartus

1.2.1. Estructura de la tabla

El componente principal del predictor de saltos propuesto consiste en la tabla que almacena el contador y el PC destino. Esta tabla fue implementada en Quartus con un módulo de memoria RAM de 64 entradas, y 64 bits por cada entrada.

Un registro de dicha tabla contiene los siguientes campos:

Unused	Contador	Valid	Tag	PC target prediction
63—59	58—57	56	55—32	31—0

El bit de **Valid** indica si el contenido almacenado en dicho registro corresponde a una instrucción de salto o no. Tenía el objetivo original de evitar leer la tabla si hubo un cambio de código, para lo cual los PC de saltos ya no se corresponderían con las entradas a la tabla. Sin embargo, en la práctica, al recompilar para cada código, la memoria se borra, por lo que este bit deja de tener utilidad, ya que siempre que se haya modificado la tabla querrá decir que contiene un salto válido.

Como se mencionó anteriormente, esta tabla se accede de manera similar a cómo se accede a una memoria caché. Se indexa a la tabla utilizando los bits menos significativos del PC (excepto los últimos dos, que serán siempre 00). Así, como se tienen 64 registros, se utilizarán los 6 bits [7:2] del PC como address a la tabla. Así, los siguientes 24 bits [31:8] conformarán el **TAG**, que permitirá identificar, al igual que en una memoria caché, que el contenido corresponderá al PC correcto.

PC target prediction posee el PC destino guardado correspondiente a la instrucción de salto de dicho registro.

1.2.2. Decisión de predicción

A la hora de realizar una predicción, se accede con el PC_next de la instrucción que será buscada en memoria. De esta manera, luego del flanco de clock, saldrá de la tabla el registro correspondiente, a la par que de la memoria principal sale la instrucción correspondiente al PC. Es decir, el predictor de saltos está en paralelo a la memoria principal, y la salida corresponderá a la instrucción que se encontrará en la etapa de Fetch, de manera tal que la predicción del salto ya estará lista para la siguiente instrucción, evitando de esta manera detener el pipeline, ya que se podrá predecir sin delays.

Al obtener el contenido del registro, se verifica (con un módulo combinacional) que el bit de valid esté encendido, que el tag del PC que se utilizó como entrada coincida con el tag del registro, y si se cumplen dichas condiciones, se evalúa el bit más significativo del contador. En caso de ser 1 (contador en 10 o 11), se activa la salida "branch prediction". En caso de ser 0 (contador 00 o 01) o de no cumplirse los chequeos anteriores, "branch prediction" será 0. A su vez, la salida "pc_target_prediction" tendrá el valor correspondiente del registro leído.

Es importante recalcar la importancia del "TAG". Si no se utilizara el tag, si bien que la predicción sea correcta o errónea en sí no afecta al correcto funcionamiento del procesador, sí afectaría el utilizar un PC destino incorrecto (es decir, saltar a una línea que no corresponde). Esto sería un error grave y llevaría a un comportamiento completamente incorrecto. Por este motivo, la utilización del TAG es esencial para asegurarse estar tomando el PC Target correcto.

1.2.3. Actualización de un registro del predictor

Para actualizar un registro, se debe haber llegado a la etapa Execute/Memory/PC Decision; en la cual ya se ha calculado si la instrucción efectivamente es un salto, si es el tipo de salto que debe ser actualizado en el predictor, y si el salto en cuestión se debería haber tomado o no.

Como se puede observar en la figura 3, la memoria RAM utilizada en Quartus nos permitió usar el puerto A para leer la memoria y realizar la predicción, mientras que el puerto B fue utilizado para realizar la actualización de la misma, a la par que se pueden realizar las lecturas para predicción. Como para modificar al contador es necesario primero leer la memoria y en el próximo ciclo de clock escribirla, esto introduce el problema de que, si vienen dos saltos consecutivos, no se podrá leer el contenido del segundo salto, ya que se estará escribiendo la actualización del primero y no hay más puertos de address de lectura. Por este motivo, este esquema tiene el problema que, en caso de ocurrir esto, no se actualiza la tabla. Esto sin embargo sólo podría desencadenar en errores de predicción o de precisión en la predicción, pero no provoca un comportamiento incorrecto. De todas maneras, la probabilidad de que caigan dos saltos consecutivos puede ser considerada pequeña.

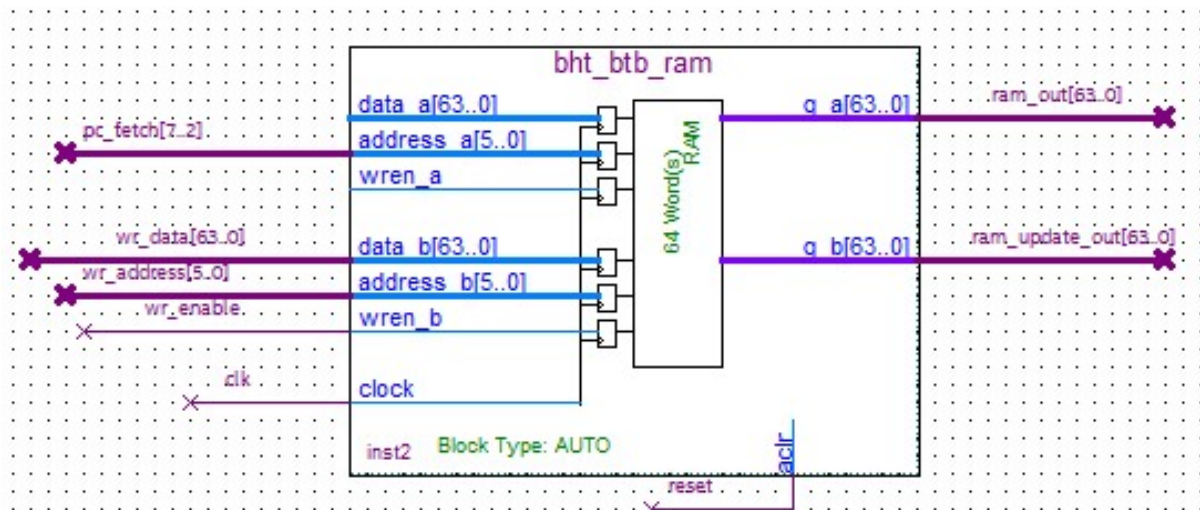


Figura 3: Bloque de RAM de la tabla

El proceso de actualización entonces consiste en un módulo que contiene una máquina de 2 estados. Se mantiene en el primer estado si se recibe la señal de que la instrucción a almacenar no es un salto o no es uno de los saltos válidos. En caso de que sea un salto válido para almacenar, accede al registro correspondiente con el PC recibido de la instrucción de salto, pasa al siguiente estado en el próximo ciclo de clock, y luego lee el contador. Si se indicó que el salto se tomó, se incrementa en 1 el contador (saturando en 11), en caso contrario, de decrementa en 1 (saturando en 00). Así, se vuelve a armar el registro, con el TAG correspondiente al PC recibido y actualizando el PC_TARGET con el valor destino también recibido, y en el próximo ciclo de clock se actualiza la tabla.

Cabe aclarar que la elección de la cantidad de 64 registros fue para poder realizar una prueba que no ocupara mucha memoria y que tuviera sentido para la longitud de los programas que íbamos a probar. Debido a que se usa la tabla como una memoria caché, podrían haber colisiones, las cuales ya sabemos que para predicciones no serían un problema, ya que simplemente se predeciría negativamente, sea correcta o no la predicción. La colisión tendrá efecto cuando la instrucción de salto llegue a la etapa de actualización, en la que este salto más reciente sobrescribirá al salto que estaba anteriormente guardado. En particular, en esta implementación sencilla no se chequea que sea igual al anterior y se reusa el contador. Una mejora sencilla podría haber consistido de leer el TAG, y de no coincidir, reiniciar el contador, o colocarlo en 01.

(Aclaración: la memoria RAM posee un Asynchronous Clear, ya que, en caso de haber un reset, todos los latches ponen en 0 sus salidas. Sin esto, el predictor almacenaría su predicción antes del reset, y podría

afectar al resultado. Con el Asynchronous Clear, se logra que al aplicar un reset, la salida del predictor sea nula).

2. Implementación detallada del proyecto

A continuación, se presentará bloque por bloque, qué señales recibe cada uno, qué procesamiento realiza, cuáles son las salidas, y cómo se integra a la lógica del procesador.

2.1. Fetch Stage

Como primera etapa, se encuentra el "Fetch". Allí, se encuentra el bloque "Fetch Unit". El mismo se encarga de almacenar el PC, de elegir el PC_NEXT y de funcionar como latch de las instrucciones de 32 bits que vienen de la memoria.

El cálculo de "pc_next" se realiza de manera combinacional. "pc_next" valdrá "pc+4" en el caso por default. En caso de recibir el bit de "take_new_pc" activo, "pc_next" tomará el valor "pc_new", el cual es calculado por un bloque previo en función de la salida del predictor de saltos, de si hubo error en la predicción anterior y de si hay que realizar algún salto.

"pc_next" se encuentra conectada al bloque de memoria, para buscar la instrucción siguiente al venir el próximo ciclo de clock.

Existe una condición adicional para el "pc_next", el cual es si se recibe una señal de "stage_ena" proveniente del "pipeline control" cuando hay que detener el pipeline. En tal caso, "pc_next" mantendrá el valor de "pc". Esto se realiza de manera tal de que, en el próximo ciclo de clock, no salga de la memoria ram la próxima instrucción. Esto se realizó, ya que la salida de la ram no posee un enable que permita retener el dato de 32 bits.

En cuanto a los procesos secuenciales: "pc_next" actualiza al valor del "pc". Las instrucciones provenientes de memoria y el bit del predictor de saltos se latching en este módulo, para permitir la propagación hacia la siguiente etapa.

Es importante aclarar que "pc" es un registro que contiene el program counter de la instrucción que acaba de salir de la memoria. Por lo tanto, este módulo también realiza un latch a este "pc" en un registro llamado "pc_dec", de manera tal de propagar el "pc" hacia la siguiente etapa de "Decode".

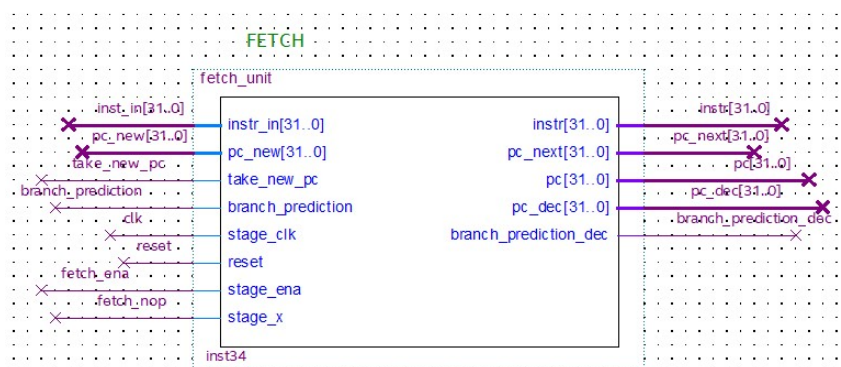


Figura 4: Bloque Fetch

2.2. Decode Stage

En la etapa de Decode, que sigue a la etapa de Fetch, consta de dos bloques: el "opcode.decode" y el "immediate.builder". Al final, se encuentra el latch correspondiente que lo separa de las etapas de "Operand" y "Address builder".

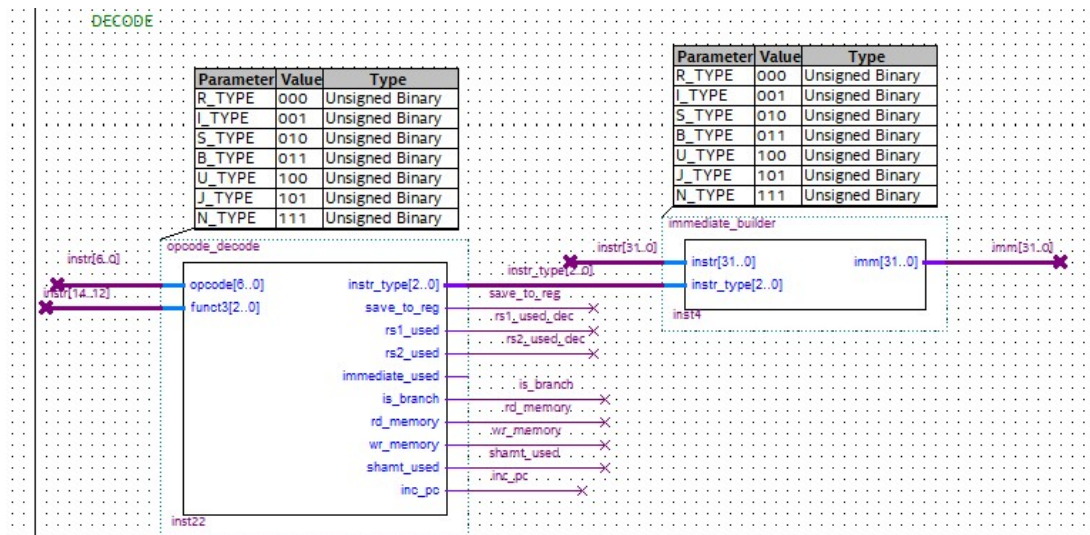


Figura 5: Módulos de Opcode Decode e Immediate Builder

2.2.1. Opcode Decode

Este módulo (combinacional) es el encargado de leer la instrucción y de generar las señales de "instr_flags" necesarias para que todos los bloques de las próximas etapas del pipeline puedan interpretar qué operación realizar. Por lo tanto, es uno de los bloques más importantes, ya que a partir de modificar el mismo, es posible agregar y modificar cómo se comportan las instrucciones.

El bloque recibe, de la instrucción de 32 bits, los últimos 7 bits (OPCODE) que indican qué tipo de operación realizar, y los bits correspondientes a "funct3". Con estos datos, es posible decodificar los principales tipos de instrucción, que comparten las mismas señales de "flag". Dichas instrucciones, para recordar, se encuentran en la tabla 6. Como se puede observar, cada "opcode" tiene, además, un formato de instrucción particular, que permite saber cómo interpretar los bits de la instrucción.

opcode	funct3	Instr Format
LUI	xxx	U
AUIPC	xxx	U
JAL	xxx	J
JALR	xxx	I
LOAD	xxx	I
OP-IMM	not[001,101]	I
OP-IMM	001 or 101	R
OP	xxx	R
STORE	xxx	S
BRANCH	xxx	B
MISC-MEM	xxx	I [N/A]
SYSTEM	xxx	I [N/A]

Tabla 2A – Formato de instrucciones
(Nota: solo se incluyen los opcodes del set RV32I reducido)

Figura 6: Tabla de los opcodes y el formato de instrucción

Como salida de este módulo, se encuentran las siguientes señales:

- 1) **"instr_type[2:0]":** Esta señal, de 3 bits, codifica qué formato de instrucción corresponde a la instrucción. Es una señal de suma importancia, ya que permite a los siguientes módulos saber qué y cómo interpretar las señales que llegan. Principalmente, saber si se debe leer "rs1", "rs2", "immediate" o "pc". Es muy utilizado por los siguientes bloques, especialmente en la etapa de "execute" y para saber el tipo de salto de que se trata.
- 2) **"save_to_reg":** indica al banco de registros que la salida de la ALU debe ser almacenada. Utilizado por el pipeline control para evitar colisiones en datos que se deben leer y no han sido almacenados aun.
- 3) **"rs1_used" y "rs2_used":** indica que la instrucción utiliza el contenido indicado en los campos de "rs1" y "rs2", del banco de registros. Es utilizado por el pipeline control para evitar colisiones en datos que se deben leer y no han sido almacenados aun.

4) **"is_branch"**: indica que es un salto. Utilizado por el predictor de saltos y por el módulo que debe decidir si debe realizar un salto o no.

5) **"rd_memory"**: es para indicar al banco de registros que debe almacenar el valor proveniente de la memoria, en vez de la salida de la ALU (o no almacenar ningún valor). También es usado por el Pipeline control para evitar conflictos con la escritura de los datos provenientes de memoria a los registros que deben ser utilizados por instrucciones futuras sucesivas.

6) **"wr_memory"**: indica a la memoria que debe almacenar el valor indicado. Se utiliza como "wr_enable".

7) **"shamt_used"**: es para indicar a la ALU que debe usar como "shamt", para operación de SHIFT, al registro "rs2". Es un caso anómalo, en que no se debe interpretar "rs2" como leer el registro "rs2", sino que hay que leerlo como un inmediato.

8) **"inc_pc"**: indica a la ALU que debe sumarle 4 al PC. Es utilizado para las instrucciones de salto que, además de calcular el address indicado, indica que se debe guardar el valor del PC+4 en el banco de registros (JAL y JALR). También se usa para AUIPC, para indicar que se trata de la instrucción tipo "U" que suma al PC. Se aprovecha el hecho de que, en nuestra arquitectura, lo indicado en "rd" se guarda en el banco de registro con la salida de la ALU si está "save_to_reg" en 1 (solo hay 2 posibles caminos para almacenar valores en el banco de registros: por la ALU o por la memoria).

2.2.2. Immediate builder

Este bloque (combinacional) recibe el "instruction_type" decodificado anteriormente, y a partir del tipo de instrucción y de los bits de la instrucción, arma los 32 bits del operando inmediato. Por defecto, si no se utiliza inmediato, lo deja en 0. Este bloque realiza todas las extensiones de signo necesarias. Se basa en la codificación según cada tipo de instrucción, como se observa en la tabla 7

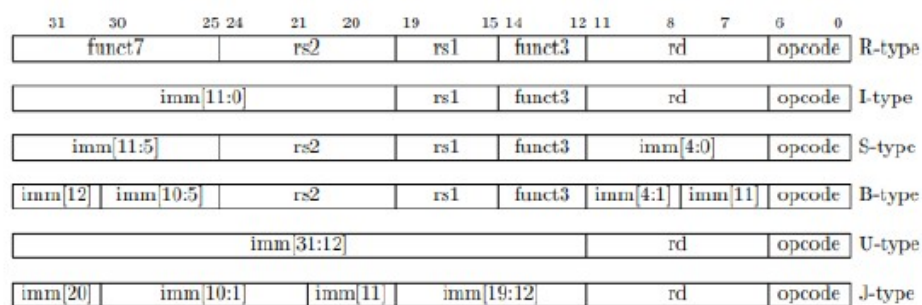


Figura 1.2 – formatos de instrucciones para RV32I

Figura 7: Tipos de instrucción y decodificación

2.2.3. Funct3, Funct7, RS1, RS2 y RD

Los valores Funct3, Funct7, RS1, RS2 y RD se decodifican directamente eligiendo los bits necesarios. No necesitan lógica adicional, ya que siempre se calculan de la misma manera al estar en la misma posición. En todo caso, no se los utilizará si la instrucción no los utiliza y, por lo tanto, sus valores carecerán de sentido. Estos valores pasan directamente al latch de salida de la etapa "Decode".

2.3. Operand y Address Builder Stage

Las etapas de Operand y Address Builder se encuentran en la misma etapa del Pipeline. Podrían considerarse una misma etapa, pero en la estructura decidimos separar sus latches de salida para claridad en la visualización de la estructura. Conceptualmente, estos latches a la salida de la etapa son controlados de la misma manera por el pipeline control; sólo difieren en las señales que almacenan y propagan a la siguiente etapa. Pueden verse como un mismo latch dividido en dos partes.

2.3.1. Banco de registros en la etapa Operand

Es conveniente, en este momento, introducir al banco de registros (ver figura 8), el cual es utilizado en la etapa de "operand" a la hora de leer o seleccionar los registros; y es utilizado en la etapa de "write

back” para actualizar los mismos.

En la etapa ”Operand”, funcionando de manera combinacional, simplemente recibe los valores de ”rs1” y ”rs2” (5 bits) que ’apuntan’ a los registros; y a la salida habilita el contenido de dichos registros, cada uno de 32 bits. Los detalles de la actualización se explicarán en la correspondiente sección de ’Write Back’.

Estos contenidos de ”rs1” y ”rs2” son potencialmente utilizados por la ALU y por el Address Builder.

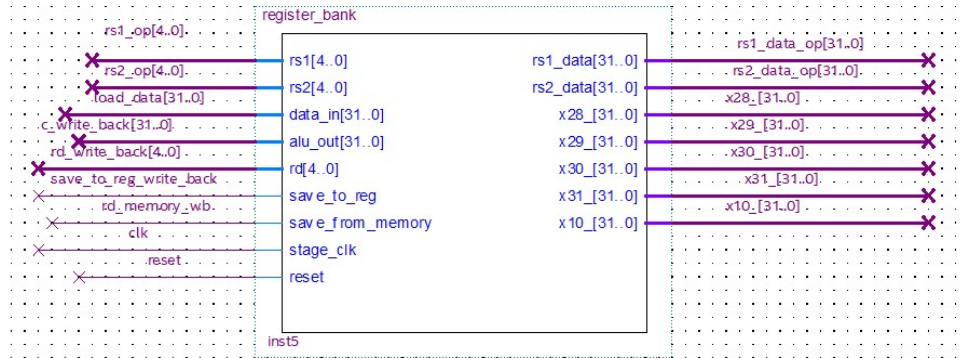


Figura 8: Bloque de Register bank

2.3.2. Address Builder

El bloque address builder (combinacional), que se observa en la figura 9, recibe instruction type y si la instrucción es un salto. En función de estos dos datos, elige qué valores de entrada (inmediato, rs1 y pc) necesita sumar para formar el address correspondiente, tanto para saltos como para accesos a memoria para lecturas o escrituras.

Además, arma un "flag_branch" para indicar el tipo de salto que se trata. '00' significa que no es un salto, '01' que se trata de un salto incondicional con inmediato, '10' que es un salto incondicional que utiliza 'rs1'; '11' indica que es un salto condicional (que usa inmediato). Este flag es necesario para indicar al predictor de saltos que debe actualizar su registro correspondiente en caso de que sea un salto que no utilice el valor de un registro (no use rs1).

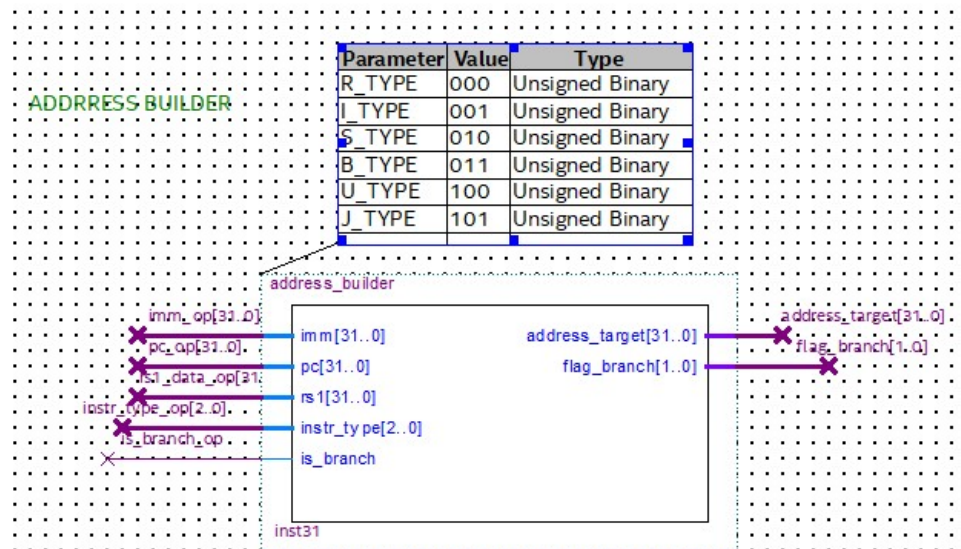


Figura 9: Bloque de Address Builder

2.4. Execute, Memory y PC update Stage

La etapa de ejecución de la Alu, el acceso a memoria para lectura o escritura; y la decisión del próximo PC (en los casos de tener que realizar un salto, o corrección de salto por error de predicción) ocurre en esta tercera etapa.

2.4.1. Execute: ALU y Operand Build

En la figura 10 se pueden observar, primero el bloque de operand build y luego la Alu, como proponía la estructura provista en la consigna.

El bloque "operand_build" (combinacional) recibe el "instruction_type" para poder decidir qué valores (rs1, rs2, inmediato o pc) utilizar en la Alu como entrada "a" y "b". Recibe además el valor del índice de "rs2" y el flag de "shamt_used" para hacer la distinción anteriormente mencionada para los shifteos con valores inmediatos; y poder utilizar dicho valor. También recibe el flag "inc_pc" para colocar en la Alu al "PC" y un "4" para que se haga dicha suma en las instrucciones que ya se mencionó que lo necesitan.

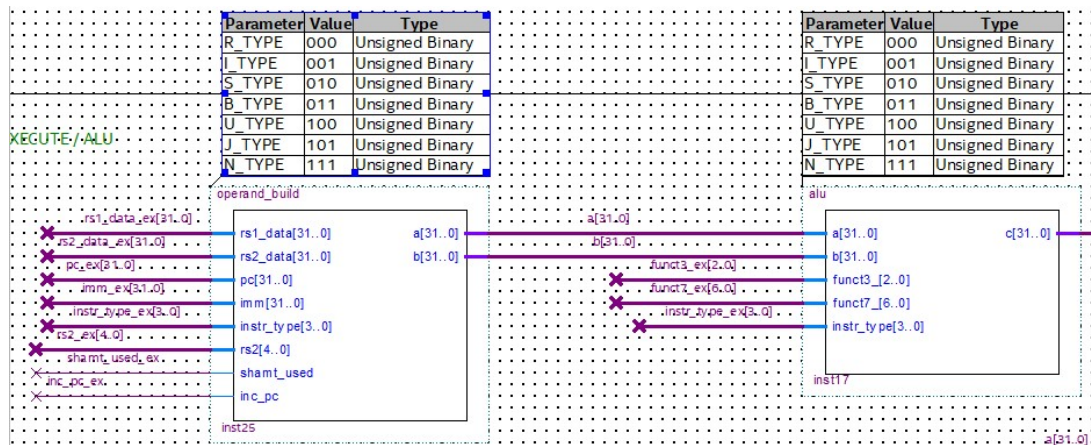


Figura 10: Bloque de Operand Build y Alu

El bloque ALU (combinacional) es el bloque que más operaciones puede realizar, y es el que termina ocasionando el cuello de botella en la frecuencia máxima de clock debido a sus tiempos de propagación.

Este bloque recibe "a" y "b", y en función de los valores de "funct_3" y "funct_7", realiza la operación correspondiente. Debido a que los distintos formatos de instrucción realizan operaciones distintas, este bloque también recibe el "instruction type" para saber qué operación realizar. En particular, esto es necesario para los saltos condicionales, ya que las operaciones de comparación la realiza la ALU; y para las instrucciones que deben sumar un valor al PC y luego almacenar en un registro.

2.4.2. Memoria: store_data_module

En la figura 11, se puede observar la memoria RAM utilizada en Quartus. Utilizamos de manera compartida la memoria con las instrucciones y datos.

Tanto para lectura o escritura, simplemente se manda el address armado en la etapa anterior, sin los 2 bits menos significativos (debido a que la memoria se accede de a 32 bits).

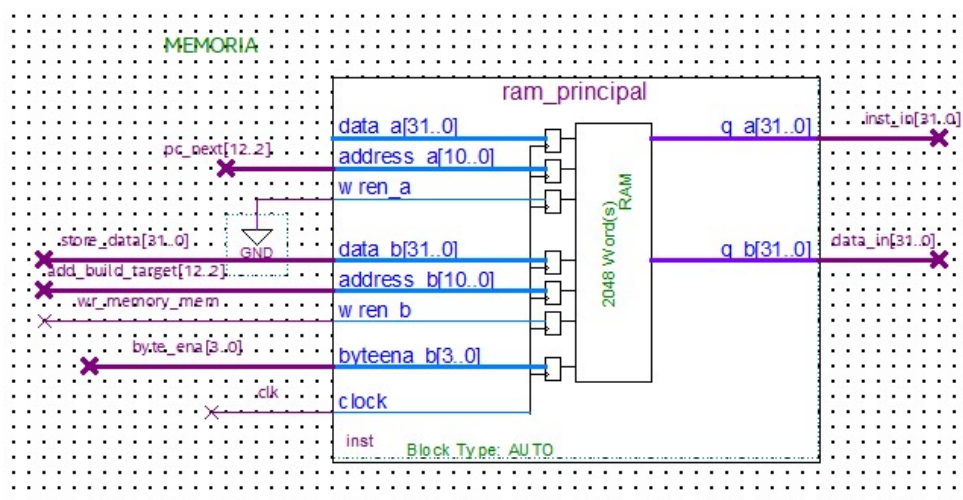


Figura 11: Bloque de Store Data

En el caso de la escritura, se implementó un módulo llamado "store_data" (combinacional), que se observa en la figura 12, para poder adaptar el dato a guardar al formato que necesita la memoria.

Recibe "funct3" para distinguir el tipo de guardado que debe realizar (8, 16 o 32 bits), y en función de eso, utiliza los dos bits menos significativos del address para decidir en qué bytes colocar los bits correspondientes, provenientes de "rs2"; que fue debidamente propagado de la etapa anterior. Para no modificar los otros bytes de la memoria RAM, se activan los bits correspondientes del "byte_enable" en cada caso.

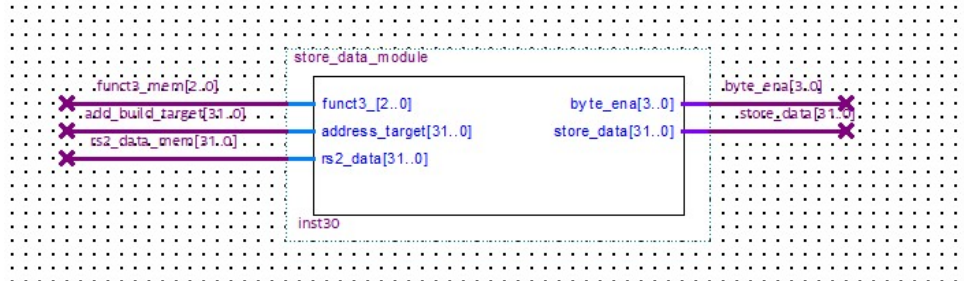


Figura 12: Bloque de Store Data

2.4.3. PC Update: Next_pc_decision_module y Branch_result

Por último, tenemos los módulos que deciden cómo debe ser actualizado el PC.

Como primer bloque, tenemos el módulo de "branch_result" (combinacional) (ver figura 13). Este bloque simplemente recibe el flag de qué tipo de salto es, y si es un salto condicional, indica que es un salto en función de la salida de la ALU, que en esta etapa está realizando las comparaciones correspondientes. Si es un salto incondicional, simplemente indica que se debe tomar el salto.

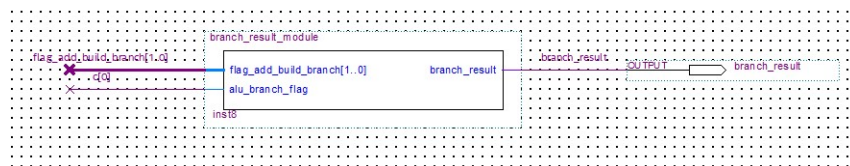


Figura 13: Bloque de Branch Result

En segundo lugar, tenemos el módulo combinacional "next_pc_decision" (ver imagen 14). Este módulo decide cuál debe ser el próximo PC en función de si el salto actual indica si hay que saltar, de la predicción previa realizada sobre la propia instrucción de salto etapas antes, y de la predicción actual del predictor de saltos. La lógica es la siguiente:

Primero se compara el resultado del salto con la predicción realizada sobre dicha instrucción de salto etapas antes. En caso de coincidir, quiere decir que la predicción del salto anterior fue correcta, por lo que no se debe realizar un "flush" del pipeline y que las instrucciones que ingresaron al pipeline fueron las correctas. Luego, para decidir el próximo PC, se observa la salida del predictor de saltos en este instante: en caso de indicar que se debe saltar, se toma el valor del "pc" que proviene del predictor; en caso contrario, simplemente se indica que se debe sumar "4" al pc actual.

Si el resultado del salto difiere con lo que había predicho el predictor de saltos, se debe hacer un "flush" del pipeline, ya que las instrucciones que ingresaron fueron incorrectas. Luego, se debe corregir el PC. Entonces, se observa el resultado del salto. Si se indicó que se debía saltar, se actualiza el PC al address calculado en el "Address Builder" que contiene el PC correcto. En caso de indicar que no se debe saltar, quiere decir que el predictor indicó que se debía saltar cuando no se debía. Por lo tanto, se le debe sumar "4" al "PC ANTERIOR". Este PC anterior, correspondiente a la instrucción de salto en cuestión, fue propagado a través de los latches.

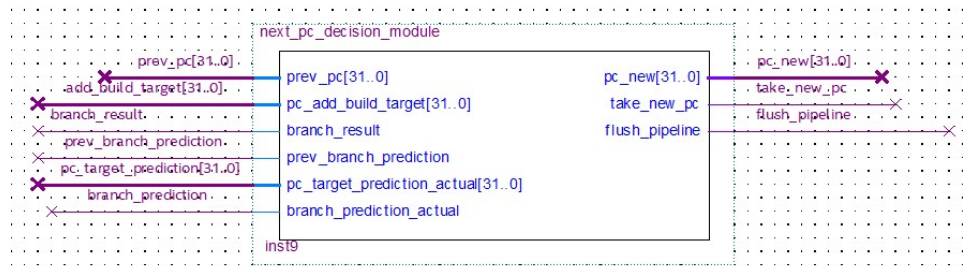


Figura 14: Bloque de next_pc_decision

2.5. Write Back

En la etapa Write Back, se decide si se debe guardar en los registros algún valor, y en caso afirmativo, si se debe almacenar los valores provenientes de la memoria o provenientes de la ALU.

2.5.1. Load Data Module

En la figura 15, se observa el módulo combinacional que interpreta correctamente los bits leídos desde la memoria. Recibe, al igual que su par de lectura, "funct3" para decidir si debe leer 8, 16 o 32 bits dependiendo de la instrucción de lectura que se trate. Luego, con los 2 bits menos significativos del address (propagado de la etapa de anterior), se eligen qué bytes se deben leer, y se realizan las extensiones de signo para las instrucciones "signed" o no se la realiza para las instrucciones "unsigned" (depende de "funct3").

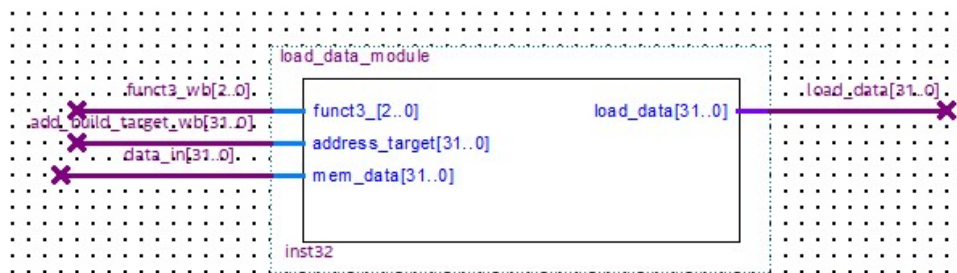


Figura 15: Bloque de Load Data

2.5.2. Banco de registros en Write Back

Por último, en la figura 16 se observa nuevamente el banco de registros. Ahora, en la etapa Write Back, se recibe "rd"; se recibe "c", la salida de la ALU de la etapa anterior; "load_data", que es la salida de la memoria ya procesada por el módulo explicado anteriormente; y los flags "save_to_reg" y "rd_memory".

En este caso, de manera secuencial, es decir, al momento del flanco de clock, se actualizará el valor del registro indicado por "rd" que fue debidamente propagado por las etapas. Si "save_to_reg" está habilitado, se almacenará el valor de la ALU; si "rd_memory" está habilitado, se almacenará el dato proveniente de memoria (tiene prioridad frente a la ALU, fue una decisión arbitraria); y si ninguno está habilitado, no se actualiza ningún registro.

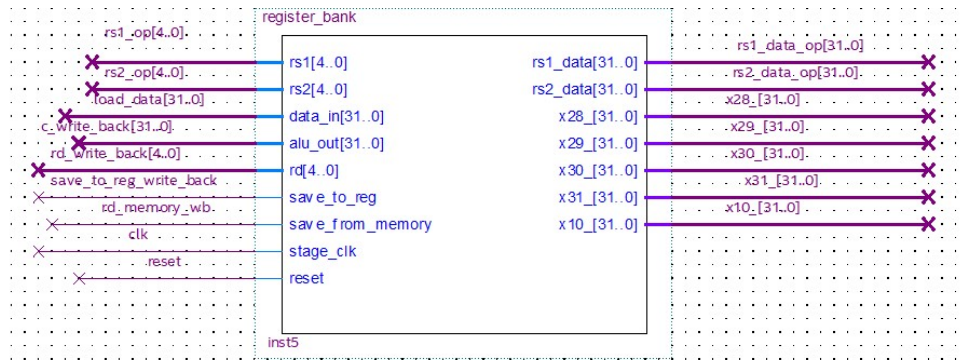


Figura 16: Bloque de registros (write back)

2.6. Pipeline Control

Finalmente, para lograr un correcto funcionamiento y sincronismo de las instrucciones, se tiene el módulo de Pipeline Control, que se observa en la figura 17. Se trata de un módulo combinacional que realiza un halt o flush del pipeline en caso de los saltos o de hazards debido a dependencias en el acceso a registros (tanto por actualización desde la Alu o desde lectura de memoria). Las señales que envía indican a cada latch si deben habilitar o deshabilitar la actualización de sus registros; o si deben limpiar sus registros con 0's, de manera de introducir NOPs en el pipeline debido a halts o flush.

Cabe aclarar que, como todas las instrucciones realizan el acceso a memoria, tanto en escritura como lectura, en la misma etapa del pipeline, no se encuentran colisiones en el acceso a memoria. Además, como no se implementaron señales de "mem_ready" debido a la naturaleza de la RAM de la D0NANO, no hizo falta analizar los casos en que no se recibió la confirmación de memoria de lectura o escritura. En caso de querer implementar esto, simplemente se utilizaría dicha señal para detener el pipeline (halt de todas las etapas) hasta obtener dicha confirmación y habilitar las etapas nuevamente.

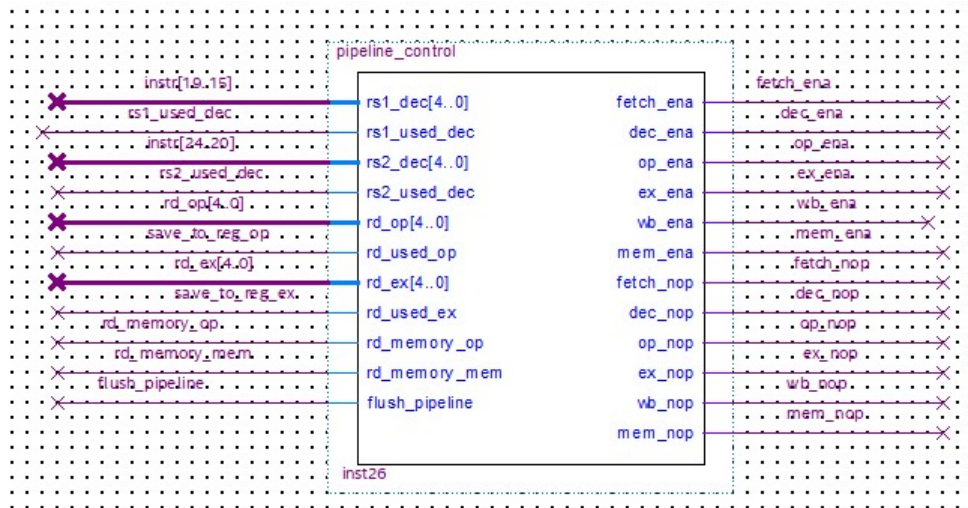


Figura 17: Módulo de Pipeline control

2.6.1. Funcionamiento del Pipeline Control

1) **Flush del pipeline (error de predicción):** Cuando recibe una señal de flush de pipeline, el pipeline control introduce NOPs en los latches de salida de las etapas de fetch, decode y operand/address_builder. Esto se debe a que la decisión de realiza el flush se realiza en la etapa de execute/memoria/pc_decision; por lo que se deben vaciar las instrucciones posteriores a la instrucción de salto, y se debe permitir que terminen las instrucciones anteriores.

2) **Dependencias en acceso a registros:** Esta dependencia ocurre cuando una instrucción, en la etapa Operand accede a registros que una instrucción anterior debía haber modificado, pero que aún no realizó la actualización.

Para evitar esto, el Pipeline Control analiza, en primer lugar, si el "rs1" o "rs2" (si se usan) en la etapa decode de una instrucción, coincide con el "rd" de la etapa Operand de la instrucción anterior (si se utiliza y es distinto de '0', pues el registro "x0" no puede ser modificado). En tal caso, se realiza HALT de las etapas de Fetch y Decode, y se inserta un NOP en la salida de la etapa Decode. Así, se retiene a la instrucción en Decode, y la instrucción con la dependencia pasa a la etapa de Execute.

En segundo lugar, el Pipeline Control analiza, si el "rs1" o "rs2" (si se usan) en la etapa decode de una instrucción, coincide con el "rd" de la etapa Execute de la instrucción dos etapas anterior. Esto se debe a que aún no ha escrito en memoria, y al próximo ciclo de clock, el resultado recién estará en la entrada al banco de registros. En tal caso, el Pipeline Control realiza un Halt de Fetch, Decode y Operand/Address; e introduce NOPs a la salida de los latches de Operand/Address.

Ya un chequeo entre instrucciones en Decode y Write Back no es necesario, ya que en el próximo ciclo de clock, se actualizarán los registros y la instrucción en Operand leerá de ellos de manera combinacional.

Es importante notar que como las actualizaciones de los registros, ya sea desde la Alu como desde datos de memoria, ocurren en la misma etapa de Write Back, no fue necesario añadir lógica adicional o distinta para ambos casos.

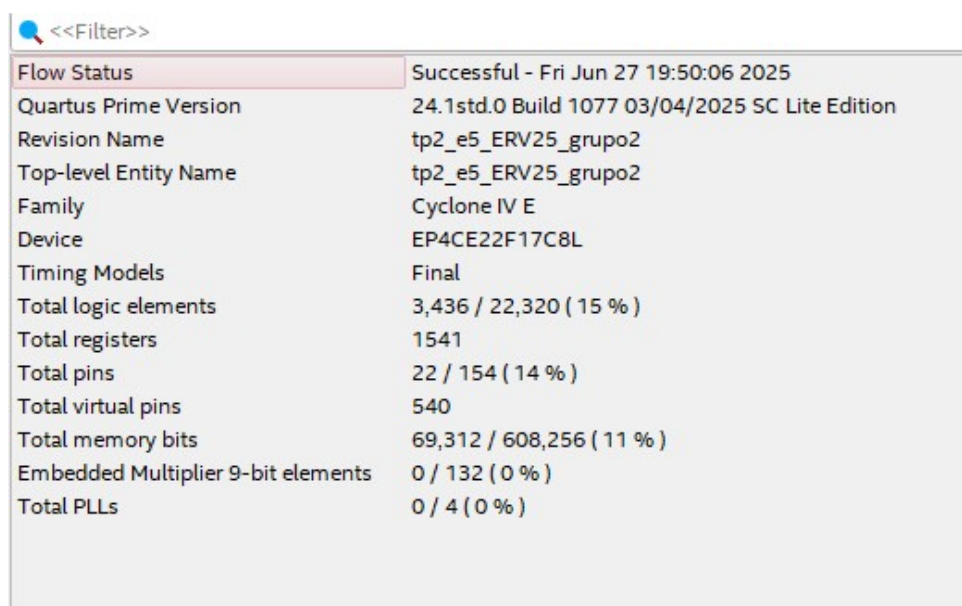
3. Simulaciones

Se programó en la página <https://riscvasm.lucasteske.dev/#>, que permite obtener los hexdecimales del código. Luego, a través de un código en Python, que se adjunta en las carpetas del proyecto con el nombre "bin_to_intel_hex.py", se pasa este hexadecimal al formato Intel Hex; en un archivo llamado "program_word.hex". Este archivo es leído por la memoria RAM principal, para inicializar sus registros y poder leer las instrucciones.

En dicho archivo de Python, se encuentran comentados todos los códigos que se utilizaron para probar cada una de las instrucciones, junto con sus respectivas representaciones hexadecimales. Si se desea probar estos códigos, simplemente se debe reemplazar en el código de Python con los valores hexadecimales correspondientes, correr el archivo de python, y compilar Quartus para que la memoria RAM se actualice con su nuevo contenido inicial.

3.1. Compilation Report

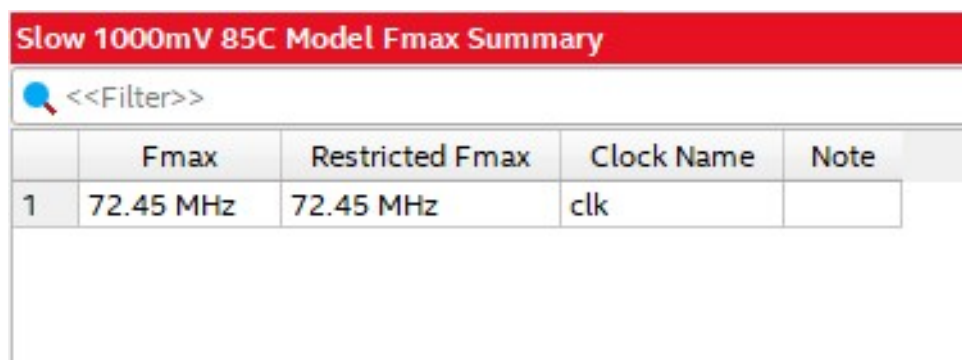
En la figura 18, se puede observar el compilation report. En el mismo, se observa que se utilizó un 15 % de los elementos lógicos.



Flow Status	Successful - Fri Jun 27 19:50:06 2025
Quartus Prime Version	24.1std.0 Build 1077 03/04/2025 SC Lite Edition
Revision Name	tp2_e5_ERV25_grupo2
Top-level Entity Name	tp2_e5_ERV25_grupo2
Family	Cyclone IV E
Device	EP4CE22F17C8L
Timing Models	Final
Total logic elements	3,436 / 22,320 (15 %)
Total registers	1541
Total pins	22 / 154 (14 %)
Total virtual pins	540
Total memory bits	69,312 / 608,256 (11 %)
Embedded Multiplier 9-bit elements	0 / 132 (0 %)
Total PLLs	0 / 4 (0 %)

Figura 18: Compilation Report

En las figuras 19 y 20, se puede observar que la frecuencia máxima estimada por Quartus para el caso más desfavorable en los tiempos de propagación es de un promedio de alrededor de 73 KHz. Por este motivo, en la práctica en la FPGA real, se utilizará el oscilador de 50 KHz que brinda la DE0-Nano. Además, en las simulaciones se utilizó también un clock de 50 KHz.



Slow 1000mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	72.45 MHz	72.45 MHz	clk	

Figura 19: Frecuencia máxima del Slow 1000mV 85C Model

Slow 1000mV OC Model Fmax Summary				
<<Filter>>				
	Fmax	Restricted Fmax	Clock Name	Note
1	74.76 MHz	74.76 MHz	clk	

Figura 20: Frecuencia máxima del Slow 1000mV OC Model

3.2. Simulación con código de Fibonacci

A continuación, se observarán los resultados de compilar el siguiente código de fibonacci (Timing Simulation), que calcula el número número 12 de dicha secuencia (el número 89):

Listing 1: Cálculo de la serie de Fibonacci en RISC-V

```

/* Fibonacci */
addi x1, x0, 0          /* x1 = 0 */
addi x2, x0, 1          /* x2 = 1 */
addi x4, x0, 10         /* x4 = 10 repeticiones */

loop:
    add x3, x1, x2       /* x3 = x1 + x2 */
    addi x1, x2, 0       /* x1 = x2 */
    addi x2, x3, 0       /* x2 = x3 */
    addi x4, x4, -1      /* decrementa contador */
    bne x4, x0, loop     /* si x4 != 0, repetir */

    addi x28, x3, 0      /* guarda resultado en x28 */

fin:
    jal x0, fin         /* loop infinito */

```

Como se puede observar, se usan los registros x1, x2, y x3 para ir almacenando los valores de la serie y el auxiliar, y se usa x4 como contador de cantidad de loops. Se guarda luego en x28 el resultado, ya que este registro estará conectado a los LEDs de la FPGA, donde se visualizará el resultado.

3.2.1. Simulación con el predictor de saltos activado

En la figura 21, se puede observar las señales correspondientes a la simulación del código de fibonacci, hacia el final del algoritmo.

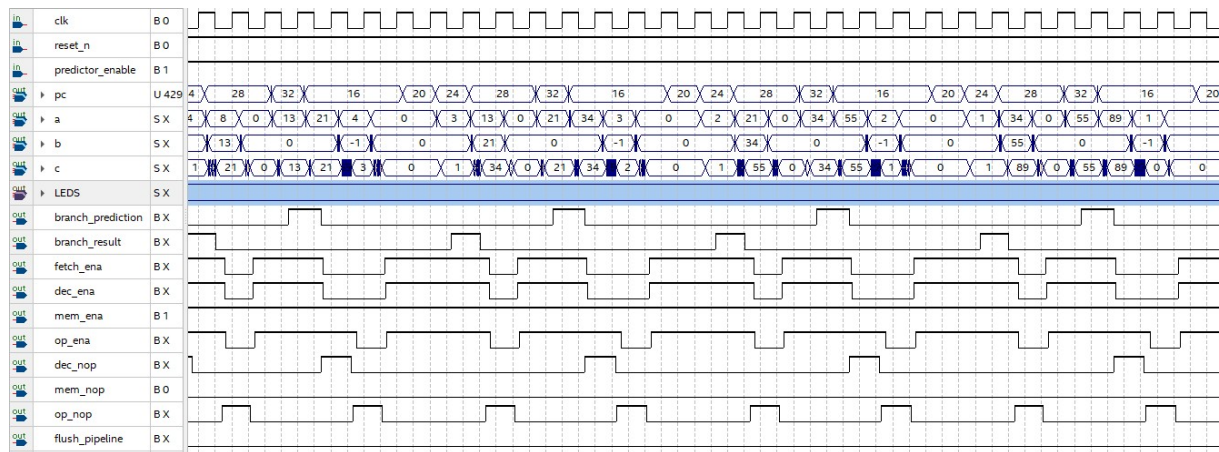


Figura 21: Señales de la simulación de Fibonacci con el Predictor de Saltos (final del código-resultado)

Se pueden observar las señales del PC, cómo van cambiando; y los registros "a" y "b" de la ALU, y el registro "c" de salida. Allí, en "c" se puede observar cómo se van realizando las sumas, y cómo llega a 89, cómo era esperado.

Se observan las señales de "branch_prediction" (la salida del predictor) y "branch_result", que indica que el salto debería tomarse. Puede verse que la señal "flush_pipeline", en este punto, está siempre inhabilitada. Esto indica que, a pesar de que en varias ocasiones se debería saltar, no se hace el salto, ya que la predicción es correcta, y no se debe hacer un flush del pipeline. También, se pueden observar las señales de 'enable' y 'nop' del pipeline. Éstas se encuentran activadas en ciertas situaciones, debido a que el código lee y actualiza registros compartidos por instrucciones sucesivas.

En la figura 22, se observa el código completo. Se ve que la señal de flush del pipeline se activa 2 veces al principio, y luego no, ya que logra adaptarse y empezar a predecir correctamente. Hacia el final, se activa algunas veces, ya que la última vez no se debe tomar el salto, y entonces realiza un error de predicción; al igual que en la última instrucción de salto incondicional para el loop infinito, en donde después de 2 ciclos logra adaptarse.

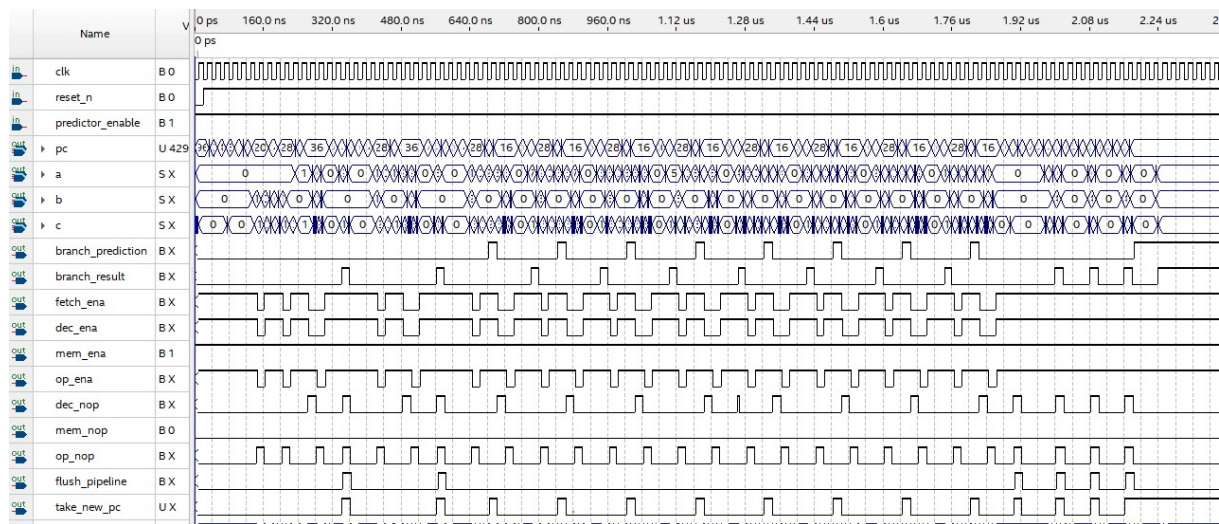


Figura 22: Señales de la simulación de Fibonacci con Predictor de Saltos (completo)

Se puede observar que tarda al rededor de $2\mu s$ con el predictor de saltos.

3.2.2. Simulación sin el predictor de saltos activado

En la figura 23, se puede observar las señales correspondientes a la simulación del código de fibonacci, hacia el final del algoritmo, sin predictor de saltos. Se puede observar que también se llega al valor 89. A su vez, se puede observar, que esta vez, está activa la señal del flush del pipeline en ciertos instantes.

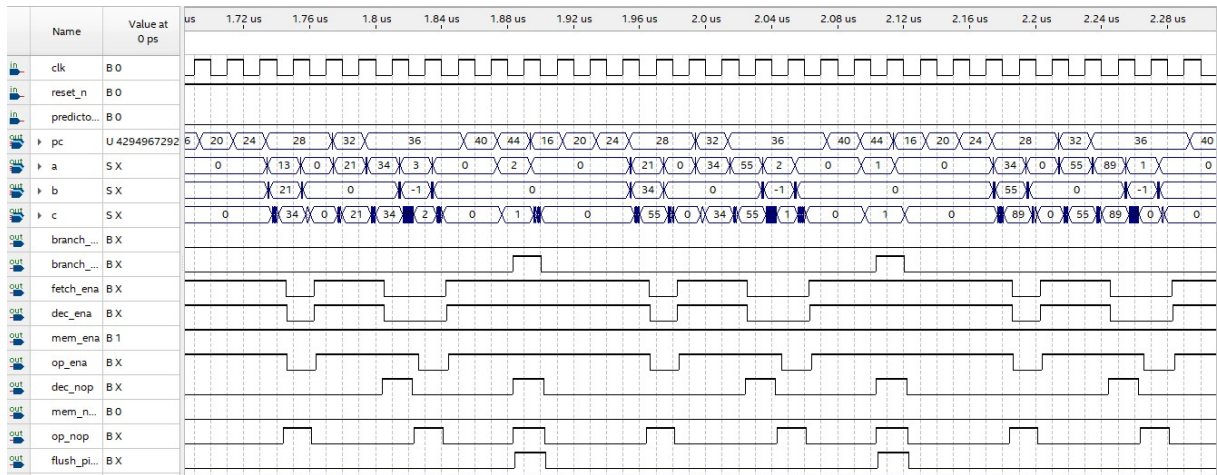


Figura 23: Señales de la simulación de Fibonacci sin predictor de saltos (final del código-resultado)

En la figura 24, se puede observar el programa completo. Se puede observar claramente la señal del flush del pipeline que se activa numerosas veces. Además, hacia el final, al no haber predictor de saltos, el salto incondicional siempre se toma 3 ciclos después, cuando se resuelve que efectivamente es un salto. Se puede observar que, en este caso, dura un poco más el programa, alrededor de 2,28 μs .

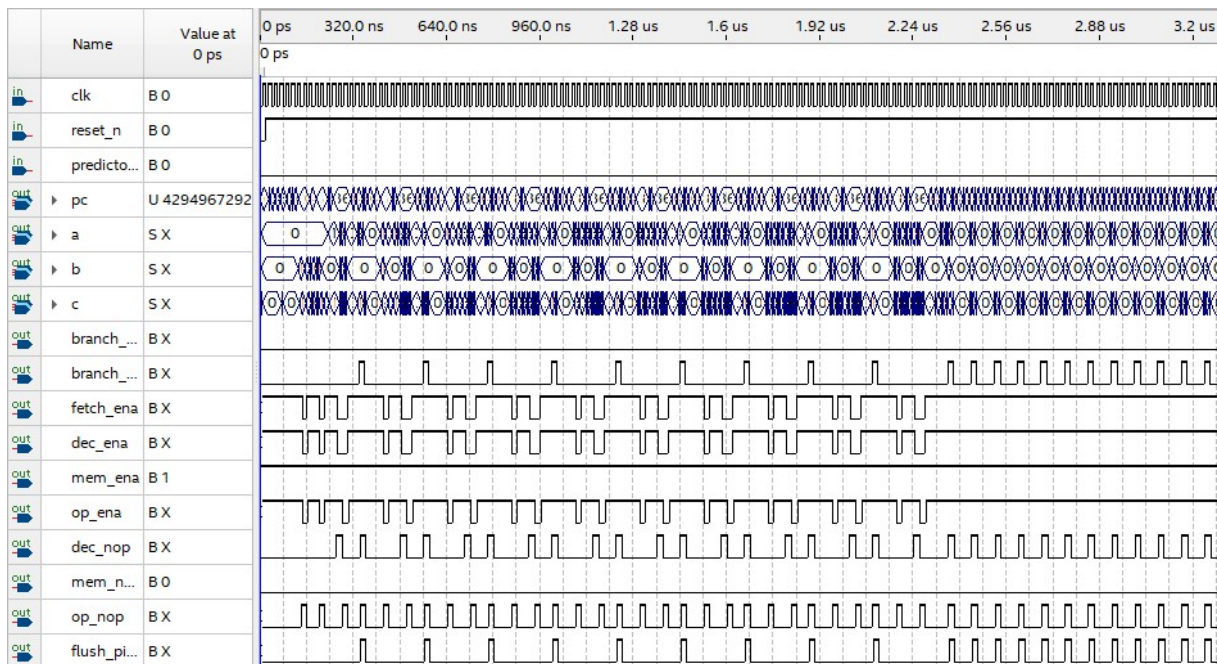


Figura 24: Señales de la simulación de Fibonacci sin predictor de saltos (completo)

En conclusión, se puede observar el predictor de saltos funcionando correctamente, y la mejora en el tiempo para este sencillo código.

4. Prueba en FPGA

Finalmente, mostraremos el código que utilizamos para ser implementado en la FPGA. Se conectaron los 8 bits menos significativos del registro $x28$ hacia los leds, de manera tal de observar en ellos el resultado del programa. Se conectó el registro $x10$ para obtener pines que sirvieran de stop a la hora de tomar las mediciones de la señal, y para medir la duración del código. También, se conectó la señal del 'enable' del predictor del saltos a uno de los 'switches' de la FPGA, de manera tal de poder habilitar o deshabilitar el predictor en tiempo real. Por último, el 'reset' fue asignado a uno de los botones. Esta señal fue negada, ya que los botones son activo bajo.

Se utilizó como clock la señal del oscilador de 50 MHz.

4.1. Código de prueba: contador de números primos

Como código de prueba, se utilizó un código complejo, que poseyera numerosos saltos que se tomaran o no tomaran. Para esto, se programó un código contador de números primos hasta el número N , el cual se puede cambiar en el código. Este código se fija, para cada número de 1 a N , si los números entre 2 y $\sqrt{N}$ son divisores. En caso de serlo, no se incrementa un contador. En caso contrario, se incrementa el contador.

Para calcular cuándo el divisor no supera $\sqrt{N}$, se implementó div^2 haciendo loops con la suma del mismo div repetida div veces, y comparando con N .

Para calcular si es o no divisor, se realizó un loop con restas sucesivas de N por div , hasta llegar a un número menor que div , el cual sería el resto. Si es 0, no se incrementa el contador; si es 1, se lo incrementa.

Así, se termina obteniendo la cantidad de números primos menores o iguales a N .

El código en Assembly es el siguiente:

Listing 2: Contador de números primos en RISC-V

```

addi x10, x0, 1      # Flag para señal de inicio (bit menos significativo)
addi t0, x0, 2        # i = 2, primer numero a chequear
addi t1, x0, 1000     # N = 1000, limite superior
addi t5, x0, 0        # contador de primos encontrados

outer_loop:
    addi t2, x0, 2    # divisor = 2
    addi t3, x0, 1    # es_primo = 1

inner_loop:
    # Multiplicacion divisor * divisor por sumas
    addi t4, x0, 0    # acumulador de multiplicacion
    addi t6, x0, 0    # contador de sumas

mul_loop:
    beq t6, t2, mul_end # si t6 == divisor, termina
    add t4, t4, t2      # t4 += divisor
    addi t6, t6, 1      # t6 += 1
    jal mul_loop        # repetir

mul_end:
    blt t0, t4, check_prime # si i < divisor*divisor, detener prueba de divisores

    # Modulo i % divisor por restas sucesivas
    addi x4, t0, 0      # x4 = i

mod_loop:
    blt x4, t2, mod_end # si resto < divisor, terminar
    sub x4, x4, t2      # resto -= divisor
    jal mod_loop        # repetir

mod_end:
```

```

    beq x4, x0, not_prime # si resto == 0, no es primo

    addi t2, t2, 1        # divisor += 1
    blt t2, t0, inner_loop # mientras divisor < i
    jal check_prime

not_prime:
    addi t3, x0, 0        # es_primo = 0
    jal check_prime

check_prime:
    beq t3, x0, skip_count # si no es primo, saltar conteo
    addi t5, t5, 1         # contador de primos += 1

skip_count:
    addi t0, t0, 1        # i += 1
    ble t0, t1, outer_loop # mientras i <= N

    addi x28, t5, 0        # guardar resultado en x28
    addi x10, x0, 2        # indica fin (segundo bit en 1)

end:
    jal end               # bucle infinito

```

En el código se agregó al principio ponerle un 1 al registro $x10$ para observar cuando comienza el programa. Al final, se agregó ponerle un 2 al registro, para indicar que terminó el programa (de 1 a 0 el bit menos significativo) y para tener una señal que permita cortar la simulación (0 a 1 del segundo bit menos significativos).

4.2. Resultados para el código con el predictor de saltos activado

Se mostró en la presentación de manera presencial, que el código correctamente cuenta cantidad de números primos, a través de los leds. Se probó con $N = 30$ y $N = 1000$, dando 10 y 168 respectivamente en los leds.

Como se observa en la figura 25, y se mostró en clase, el programa tardó 52,4 ms.

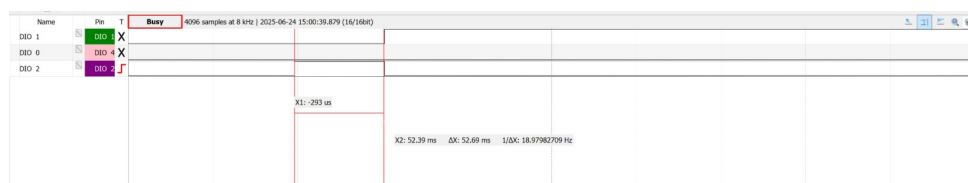


Figura 25: Resultados del programa con el predictor de saltos activados

4.3. Resultados para el código sin el predictor de saltos activado

Se mostró en la presentación de manera presencial, que el código correctamente cuenta cantidad de números primos, a través de los leds. Se probó con $N = 30$ y $N = 1000$, dando 10 y 168 respectivamente en los leds.

Como se observa en la figura 26, y se mostró en clase, el programa tardó 88,83 ms.

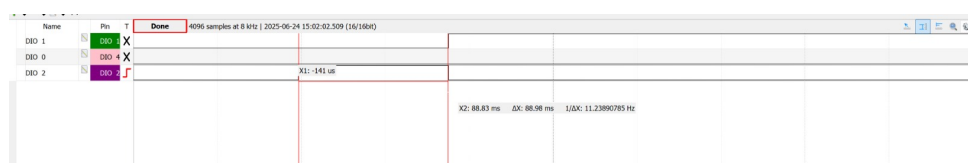


Figura 26: Resultados del programa sin el predictor de saltos activados

4.4. Conclusiones

En conclusión, en la práctica pudo observarse cómo el predictor de saltos mejoró notoriamente el tiempo del programa del contador de números primos. El tiempo de ejecución mejoró un 41 % al utilizar el predictor de saltos, que sin utilizarlo.